GA

Program

```
# Simple Genetic Algorithm
# Goal: Find a number between 0 and 10
# that gives the highest x^2 value

import random

# Fitness function
def fitness(x):
    return x ** 2

# Step 1: Create an initial population
population = [random.randint(0, 10) for _ in range(6)]

print("Initial Population:", population)

# Repeat the process for 5 generations
for generation in range(5):

    print("\nGeneration:", generation + 1)

    # Step 2: Arrange numbers based on fitness
    # Higher fitness comes first
    population.sort(key=fitness, reverse=True)

    print("Population:", population)
```

```
print("Fitness values:", [fitness(x) for x in population])

# Step 3: Select the two best parents
parent1 = population[0]
parent2 = population[1]

print("Selected Parents:", parent1, parent2)

# Keep the best parents
new_population = [parent1, parent2]

# Step 4: Create children
while len(new_population) < 6:

    # Crossover:
    # Take the average of the parents
    child = (parent1 + parent2) // 2

    # Step 5: Mutation:
    # Randomly increase or decrease the child
    mutation = random.choice([-1, 0, 1])
    child = child + mutation

    # Keep child between 0 and 10
    child = max(0, min(10, child))

    new_population.append(child)
```

```
    # Step 6: Replace old population
    population = new_population

# Find the final best solution
population.sort(key=fitness, reverse=True)

best_value = population[0]

print("\nFinal Best Value:", best_value)
print("Final Best Fitness:", fitness(best_value))
```

output

```
Initial Population: [2, 4, 0, 4, 1, 9]

Generation: 1
Population: [9, 4, 4, 2, 1, 0]
Fitness values: [81, 16, 16, 4, 1, 0]
Selected Parents: 9 4

Generation: 2
Population: [9, 6, 6, 6, 5, 4]
Fitness values: [81, 36, 36, 36, 25, 16]
Selected Parents: 9 6

Generation: 3
Population: [9, 8, 8, 7, 6, 6]
Fitness values: [81, 64, 64, 49, 36, 36]
Selected Parents: 9 8
```

Generation: 4

Population: [9, 8, 8, 8, 8, 8]

Fitness values: [81, 64, 64, 64, 64, 64]

Selected Parents: 9 8

Generation: 5

Population: [9, 9, 9, 9, 8, 7]

Fitness values: [81, 81, 81, 81, 64, 49]

Selected Parents: 9 9

Final Best Value: 10

Final Best Fitness: 100

Hypothesis Space Search

Program

```
# Simple Hypothesis Space Search
# Goal: Find the best rule for predicting
# whether a person will play outside

# Training data
data = [
    ("Sunny", "Yes"),
    ("Rainy", "No"),
    ("Cloudy", "Yes"),
    ("Sunny", "Yes"),
    ("Rainy", "No")
]
```

```
# Hypothesis 1
def always_yes(weather):
    return "Yes"

# Hypothesis 2
def always_no(weather):
    return "No"

# Hypothesis 3
def sunny_only(weather):
    if weather == "Sunny":
        return "Yes"
    else:
        return "No"

# Hypothesis 4
def not_rainy(weather):
    if weather != "Rainy":
        return "Yes"
    else:
        return "No"

# Store all possible hypotheses
hypotheses = {
    "Always Yes": always_yes,
    "Always No": always_no,
    "Sunny Only": sunny_only,
    "Not Rainy": not_rainy
}
```

```
best_hypothesis = None
best_accuracy = 0

# Search through all hypotheses
for name, rule in hypotheses.items():

    correct = 0

    print("\nTesting Hypothesis:", name)

    for weather, actual_answer in data:

        predicted_answer = rule(weather)

        print(
            "Weather:", weather,
            "| Actual:", actual_answer,
            "| Predicted:", predicted_answer
        )

        if predicted_answer == actual_answer:
            correct += 1

    # Calculate accuracy
    accuracy = correct / len(data)

    print("Correct Predictions:", correct)
    print("Accuracy:", accuracy * 100, "%")
```

```
    # Store the best hypothesis
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_hypothesis = name

print("\nBest Hypothesis:", best_hypothesis)
print("Best Accuracy:", best_accuracy * 100, "%")
# Testing Hypothesis: Always Yes
# Accuracy: 60.0 %

# Testing Hypothesis: Always No
# Accuracy: 40.0 %

# Testing Hypothesis: Sunny Only
# Accuracy: 80.0 %

# Testing Hypothesis: Not Rainy
# Accuracy: 100.0 %

# Best Hypothesis: Not Rainy
# Best Accuracy: 100.0 %
```

OUTPUT

```
Testing Hypothesis: Always Yes
Weather: Sunny | Actual: Yes | Predicted: Yes
Weather: Rainy | Actual: No | Predicted: Yes
Weather: Cloudy | Actual: Yes | Predicted: Yes
Weather: Sunny | Actual: Yes | Predicted: Yes
Weather: Rainy | Actual: No | Predicted: Yes
```

Correct Predictions: 3

Accuracy: 60.0 %

Testing Hypothesis: Always No

Weather: Sunny | Actual: Yes | Predicted: No

Weather: Rainy | Actual: No | Predicted: No

Weather: Cloudy | Actual: Yes | Predicted: No

Weather: Sunny | Actual: Yes | Predicted: No

Weather: Rainy | Actual: No | Predicted: No

Correct Predictions: 2

Accuracy: 40.0 %

Testing Hypothesis: Sunny Only

Weather: Sunny | Actual: Yes | Predicted: Yes

Weather: Rainy | Actual: No | Predicted: No

Weather: Cloudy | Actual: Yes | Predicted: No

Weather: Sunny | Actual: Yes | Predicted: Yes

Weather: Rainy | Actual: No | Predicted: No

Correct Predictions: 4

Accuracy: 80.0 %

Testing Hypothesis: Not Rainy

Weather: Sunny | Actual: Yes | Predicted: Yes

Weather: Rainy | Actual: No | Predicted: No

Weather: Cloudy | Actual: Yes | Predicted: Yes

Weather: Sunny | Actual: Yes | Predicted: Yes

Weather: Rainy | Actual: No | Predicted: No

Correct Predictions: 5

Accuracy: 100.0 %

Best Hypothesis: Not Rainy

Best Accuracy: 100.0 %